

AgentLens - AI 编程 Agent 实时监控与架构地图

完整技术设计说明书 & 开发者实施规范 (Technical Specification v1.0)

目标读者: AI 编码 Agent / Full-Stack Eng

适用的 Agent: Claude Code / Codex / Roo Code

当前版本: v1.0.0 (Production Ready)

1. 项目背景与需求定义 (Motivation & Requirements)

1.1 用户痛点分析 (User Pain Points)

在使用 Claude Code / Codex 等自主 AI 编程 Agent 时，开发者面临严重的“黑盒化”问题：

- 宏观视角缺失：** 终端（Terminal）中充斥着大量的底层微观输出（如频繁的 `cat`、`grep`、小范围文件 diff），开发者无法清晰看到 Agent 的整体 Task 任务清单、当前进展及剩余步骤。
- 死循环风险：** Agent 在遭遇测试失败或代码逻辑错误时，极易陷入“修改 -> 报错 -> 再修改 -> 再报错”的死循环，消耗大量 Token 并浪费时间。
- 卡死与假死不透明：** 当 Agent 在等待隐式终端交互、后台命令阻塞或 LLM 响应卡顿时，用户无法准确区分是“正在进行长推理”还是“系统死机”，产生严重的确定性焦虑。

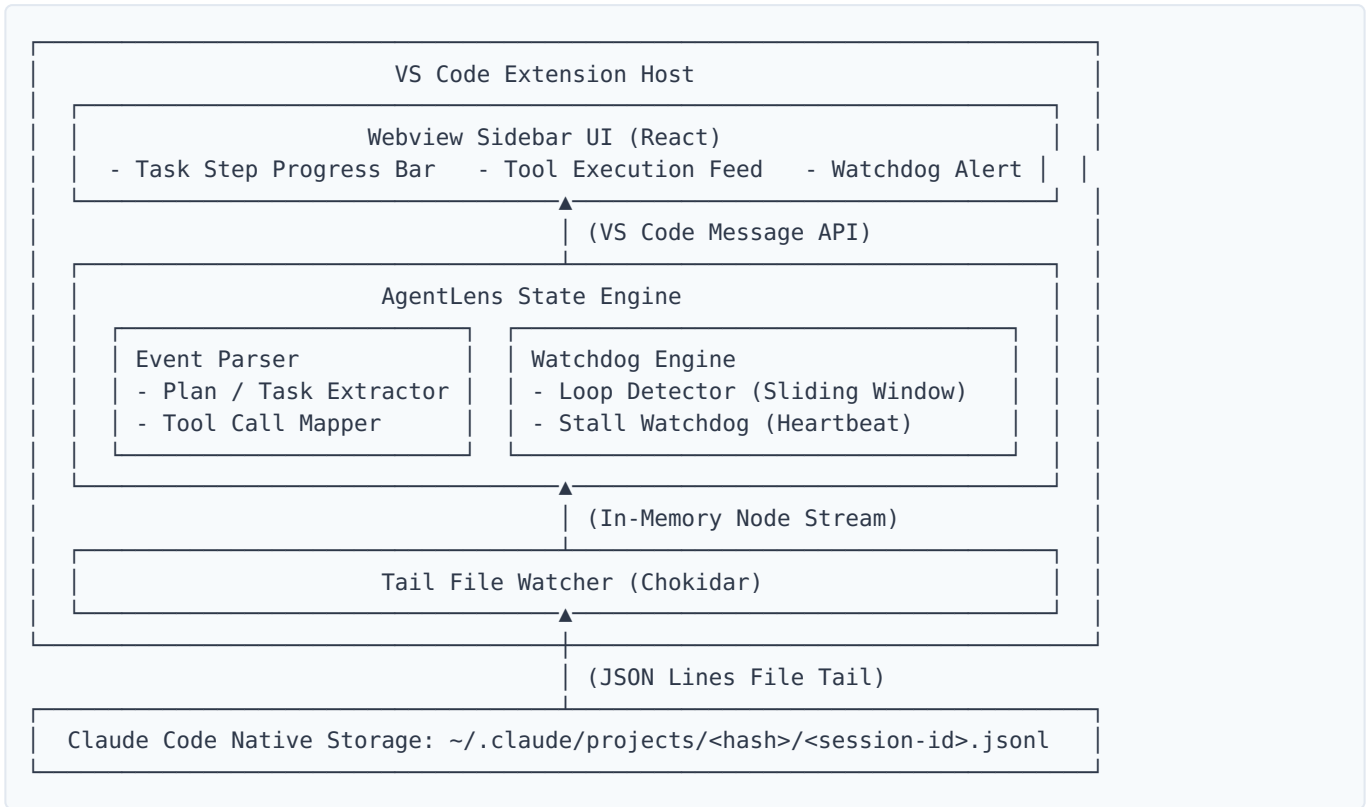
1.2 项目终极目标 (Project Goals)

构建 AgentLens（VS Code 插件 + 本地监控守护进程），分为两个递进阶段：

- Phase 1 (实时状态与死循环监控器)：** 实时捕获 Agent 的 Task 规划、映射已完成/执行中/待执行的步骤进度条，同步运行死循环检测算法（Loop Detector）与卡死预警（Stall Watchdog）。
- Phase 2 (全局架构地图与项目全貌图谱)：** 结合代码库 AST 语法树解析，绘制项目模块依赖图谱，并在图谱上实时高亮 AI 正在动刀的模块，回放跨文件修改轨迹。

2. 总体架构设计 (Overall System Architecture)

为了保证极低的系统侵入性和对 Claude Code 的零干扰，AgentLens 采用 **日志尾部事件流监听架构 (Tail File Watcher Architecture)**。Claude Code 会将所有思考链、工具调用（Tool Calls）、任务规划实时持久化到本地 `~/.claude/projects/<project-hash>/<session-id>.jsonl` 中。



3. 数据模型与接口定义 (Data Models & Interfaces)

任何 AI 或开发者可以直接复制以下 TypeScript 类型定义用于项目构建：

```
// src/types/agent.ts

export type TaskStatus = 'pending' | 'in_progress' | 'completed' | 'failed';

export interface SubTask {
  id: string;
  title: string;
  status: TaskStatus;
  startedAt?: number;
  completedAt?: number;
  relatedFiles?: string[];
}

export interface ToolCallLog {
  id: string;
  toolName: 'Edit' | 'Bash' | 'View' | 'Glob' | 'Grep' | 'Write' | string;
  summary: string;
  filePath?: string;
  command?: string;
  timestamp: number;
  status: 'running' | 'success' | 'error';
}

export interface WatchdogStatus {
  isNormal: boolean;
  loopDetected: boolean;
  stallDetected: boolean;
  loopConfidence: number; // 0.0 - 1.0
  lastHeartbeat: number;
  warningMessage?: string;
}

export interface AgentSessionState {
  sessionId: string;
  projectName: string;
  startTime: number;
  lastUpdateTime: number;
  currentTaskIndex: number;
  tasks: SubTask[];
  activeToolCall?: ToolCallLog;
  recentTools: ToolCallLog[];
  watchdog: WatchdogStatus;
}
```

4. 核心功能引擎算法规范 (Algorithmic Specifications)

4.1 死循环检测算法 (Loop Detector Algorithm)

死循环识别核心逻辑

死循环常见于连续针对同一文件尝试多次失败修补，或反复执行同一个产生错误的终端命令。

算法采用**定长滑动窗口 + 行为签名散列 (Behavior Signature Hashing)** 匹配：

1. 对于每个被解析出的 Tool Call，生成其行为签名 `Sig = Hash(ToolName + NormalizedPath/Command)`。
2. 维持一个容量 $N = 10$ 的循环队列 `Window`。
3. **模式 A (单动作反复)**: 若连续 4 次出现相同的 `Sig`（如连续针对 `auth.ts` 执行相同的 Edit 动作），触发 Level-1 死循环告警。
4. **模式 B (双动作死锁)**: 提取 `Window` 中的二元组（如 `[Edit(file.ts) -> Bash(npm test)]`），若该二元组连续重复出现 ≥ 3 次，触发 Level-2 高危死循环告警。

```
// 签名生成与判定伪代码
function generateSignature(tool: ToolCallLog): string {
  const target = tool.filePath || tool.command || '';
  const cleanTarget = target.trim().toLowerCase().replace(/\s+/g, ' ');
  return `${tool.toolName}:${cleanTarget}`;
}

function checkLoop(window: ToolCallLog[]): { isLoop: boolean; reason?: string } {
  if (window.length < 4) return { isLoop: false };
  const sigs = window.map(generateSignature);

  // Pattern A Check
  const last4 = sigs.slice(-4);
  if (new Set(last4).size === 1) {
    return { isLoop: true, reason: `连续 4 次重复执行相同工具: ${last4[0]}` };
  }

  // Pattern B Check (A -> B -> A -> B -> A -> B)
  if (window.length >= 6) {
    const last6 = sigs.slice(-6);
    if (last6[0] === last6[2] && last6[2] === last6[4] &&
        last6[1] === last6[3] && last6[3] === last6[5] &&
        last6[0] !== last6[1]) {
      return { isLoop: true, reason: `检测到死循环震荡模式: ${last6[0]} <-> ${last6[1]}` };
    }
  }

  return { isLoop: false };
}
```

4.2 卡死/僵尸进程监视器 (Stall Watchdog Algorithm)

- 定时器每 3 秒检查一次 `now() - state.lastUpdatedTime`。
- **T > 45 秒 (Yellow Warning)**: 界面提示“Agent 正在处理高耗时推理或大型编译...”。
- **T > 120 秒 (Red Warning)**: 界面亮起卡死红灯，并弹出 VS Code 通知窗口：“Claude Code 超过 2 分钟无响应，可能等待终端输入或进程阻塞。”，提供一键重启或终止选项。

5. 完整代码目录结构 (Complete Directory Tree)

AI 开发者应按照以下文件架构初始化代码库：

```

agentlens/
├─ package.json
├─ tsconfig.json
├─ webpack.config.js
├─ README.md
├─ media/
│  └─ icon.svg
│  └─ webview.css
└─ main.js (Compiled UI)
├─ src/
│  └─ extension.ts # 插件入口点 & 命令注册
│  └─ common/
│     └─ constants.ts # 超时阈值、配置常量
│     └─ utils.ts # 路径 Hash、正则助手
│  └─ watcher/
│     └─ logFinder.ts # 定位最新 ~/.claude/projects/*.jsonl
│     └─ tailStream.ts # Chokidar 增量行读取器
│  └─ parser/
│     └─ jsonlParser.ts # 原始 JSONL 转 AgentEvent
│     └─ taskExtractor.ts # Plan/Tool 转化为 TaskNode[]
│  └─ watchdog/
│     └─ loopDetector.ts # 滑动窗口死循环算法
│     └─ stallWatchdog.ts # 3s 心跳卡死检测器
│  └─ state/
│     └─ stateStore.ts # 内存单例 State Machine & Observer
└─ views/
   └─ sidebarProvider.ts # VS Code WebviewViewProvider 实现
   └─ ui/
      └─ App.tsx # Webview 内部 React / Vanilla UI 源码
      └─ components/
         └─ TaskTree.tsx # 任务步骤树与进度条
         └─ ToolFeed.tsx # 实时 Tool 执行流
         └─ WatchdogBanner.tsx # 告警与心跳指示器
      └─ index.tsx

```

6. 开发路线图与落地步骤 (Implementation Roadmap)

AI 开发者落地指南

拿到本文档的 AI 编程助手应严格按照以下顺序顺序编码：

阶段	核心任务	验收标准 (Definition of Done)
Step 1	实现 <code>logFinder.ts</code> 与 <code>tailStream.ts</code>	启动插件时能自动找到最新的 <code>.jsonl</code> 文件，并在终端成功 <code>console.log</code> 输出每一行新追加的 JSON。
Step 2	实现 <code>jsonlParser.ts</code> 与 <code>taskExtractor.ts</code>	能正确将 Claude Code 的 <code>plan</code> 和 <code>tool_use</code> 解析为带有状态的 <code>SubTask[]</code> 。
Step 3	集成 <code>loopDetector.ts</code> 与 <code>stallWatchdog.ts</code>	人为模拟连续重复工具调用或停止更新 120 秒，验证能正确产生告警对象。
Step 4	开发 <code>sidebarProvider.ts</code> 与 Webview 视图	VS Code 左侧侧边栏能渲染出漂亮的进度条、步骤卡片及实时心跳动画。
Step 5 (Phase 2 预留)	项目 AST 图谱与模块映射接口	支持读取项目目录生成 Simple Dependency Tree，并在 Webview 中展示动态高亮节点。